Arquivos, dados e APIs

Samir A. M. Martins

Universidade Federal de São João del-Rei
Departamento de Engenharia Elétrica
São João del-Rei, MG – Brasil

Graduação em Engenharia Elétrica

Roteiro

1. Lendo Arquivos
2. Dados da Internet
3. APIs
4. Encerramento

Objetivos da aula

- Entender o papel da aquisição de dados em problemas de Machine Learning.
- Ler dados locais de medição, supervisão e ensaio em CSV, TXT/log e JSON.
- Extrair dados de fontes confiáveis, setoriais, acadêmicas e comunitárias.
- Encontrar datasets e APIs para demanda, geração, máquinas, sensores e aplicações em Engenharia.

O dado como elo entre problema e modelo

- A qualidade da análise depende da origem, da estrutura e da validação do dado.
- A obtenção dos dados é a etapa central do trabalho analítico.
- Nesta aula: dados locais, arquivo remoto, API simples e busca de fontes para Machine Learning.

O que torna uma fonte confiável?

Checklist mínimo

- Instituição responsável claramente identificada.
- Documentação técnica do dado ou da API.
- Informação sobre unidades, cobertura temporal e atualização.
- Licença de uso e estabilidade do endereço.
- Possibilidade de reproduzir a coleta no futuro.

Regra prática

Prefira APIs e arquivos estruturados oficiais. Web Scraping deve ser a exceção, não a primeira escolha.

Dados para Machine Learning

- O dado pode nascer no próprio experimento ou ser buscado fora dele.
- Portais setoriais (ONS, ANEEL, ...) são fortes em carga, geração, mercado e expansão.
- Repositórios como OpenML, Kaggle, NASA e Mendeley possuem dados dos mais variados que podemos imaginar.
- Metabuscadores ajudam quando não sabemos em qual portal procurar primeiro.

Dados para Machine Learning

Fontes de dados para Machine Learning/Engenharia Eletrica

Medicao propria
Bancada
SCADA
Sensores e logs

Repositorios
OpenML
Kaggle
NASA
Mendeley Data

Portais setoriais
ONS
ANEEL
dados.gov.br
EIA

Metabuscadores
Google Dataset Search
artigos
DOIs
referencias

Exemplos reais para começar a busca

Portal	Exemplo de uso	Aplicação em ML
NASA Open Data	*Randomized Battery Usage (Random Walk)*	Prognóstico de baterias e estimação de degradação
Mendeley Data	*AC Motor Testbed under constant or varying speed*	Vibração, corrente e torque para diagnóstico de falhas
Kaggle	*Electrical System Fault Diagnosis Dataset*	Classificação de falhas a partir de correntes e resistências
OpenML	*electricity* e coleções similares	Benchmarks tabulares para classificação e mudança de regime

Principais tipos de fonte

Fonte	Vantagem	Risco comum	Quando usar
Medição local	Controle total do experimento	Erros de formato e unidade	Bancadas, osciloscópios, exportações SCADA
CSV	Acesso simples local ou por URL	Mudança de cabeçalho	Dados disponíveis em bases abertas
API JSON	Estrutura rica e documentada	Limites, filtros e mudanças de cabeçalhos	Quando o dado precisa ser atual e centralizado para compartilhamento.
HTML	Disponível em navegadores	Estrutura instável	Apenas se não houver API, CSV ou download oficial

Como encontrar datasets sem se perder

Processo recomendado

1. Defina o problema: previsão de carga, falhas em motores, qualidade de energia, manutenção preditiva;
2. transforme isso em palavras-chave de busca;
3. procure primeiro em fontes oficiais, acadêmicas ou bem documentadas;
4. verifique licença, documentação, unidades, frequência amostral e atualização;
5. registre onde encontrou o dataset e por que ele faz sentido para o problema.

Lendo Arquivos

Formatos que aparecem o tempo todo

Formato	Estrutura	Ponto forte	Ponto de atenção
CSV	Tabela retangular	Simples e universal	Separador, decimal, datas e unidade
TXT/log	Texto semi-estruturado	Flexível para eventos	Exige padronização manual
JSON	Hierárquico	Representa metadados e APIs	Normalização para análise tabular

Exemplo local: leituras de bancada

Instante	I (A)	V (V)	Vib. (m/s^2)	Temp. (°C)	Estado
08:00	10.2	219.8	1.80	34.1	normal
08:30	11.2	219.5	2.65	36.1	atencao
09:00	12.0	218.7	3.80	38.6	critico
09:10	11.6	218.8	3.10	—	critico
09:30	10.6	219.4	2.05	35.1	normal

- Exemplo de bancada motor-gerador com inversor: corrente, tensão, vibração e temperatura.
- Mesmo um CSV simples já traz problemas reais: valor ausente, data e unidades.

Leitura defensiva com pandas

```
df = pd.read_csv(
    "dados/leituras_bancada.csv",
    parse_dates=["instante"],
    na_values=["", "NA"]
)
df["potencia_VA"] = df["corrente_A"] * df["tensao_V"]
```

- `parse_dates`: evita tratar tempo como texto.
- `na_values`: explicita valores faltantes.
- Opcional: criação de uma variável derivada.

JSON aninhado e normalização

```
{
  "bancada": "B-07",
  "sensores": [
    {"id": "T01",
     "grandeza": "
        temperatura_C"},
    {"id": "I01",
     "grandeza": "
        corrente_A"}
  ]
}
```

```
df = pd.json_normalize(
    sensores,
    record_path=["sensores"],
    meta=["bancada"]
)
```

- APIs retornam JSON com alta frequência.
- Para análise, quase sempre transformamos a estrutura hierárquica em tabela.

Armadilhas frequentes na leitura

Problema	Sintoma	Mitigação
Codificação	Acentos quebrados	Fixar UTF-8 ou detectar antes de ler
Separador	Colunas coladas em uma só	Conferir `sep`, decimal e cabeçalho
Datas	Ordenação errada	Converter para `datetime`
Valores ausentes	Média e gráficos distorcidos	Definir marcadores de falta e validar
Unidades	Comparação enganosa	Registrar grandeza e unidade desde a coleta

Dados da Internet

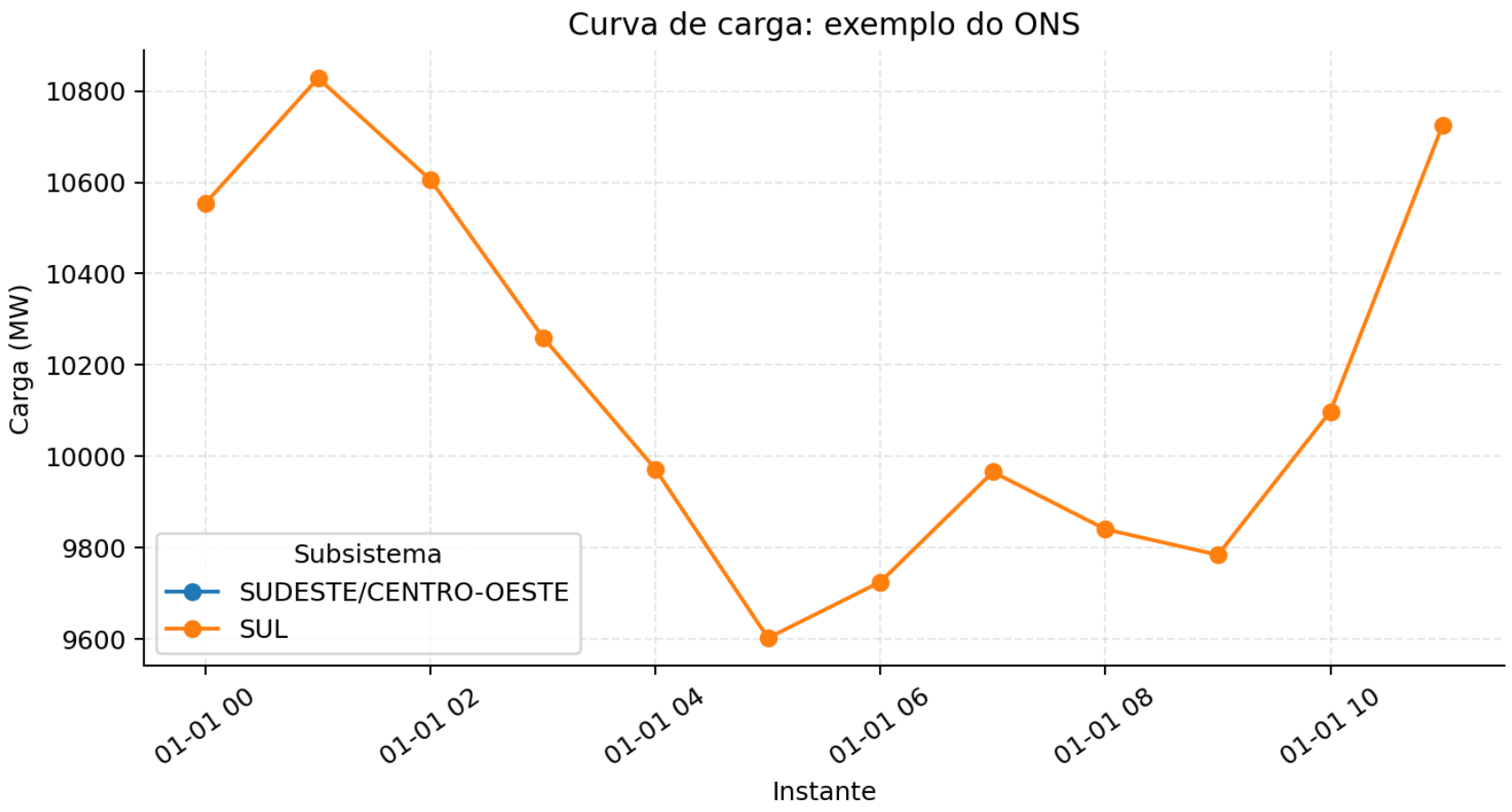
Sem complicação:

Caso	Comando mínimo	Quando usar
Arquivo local	`pd.read_csv(...)`	Dados já salvos no seu PC
Arquivo remoto	`pd.read_csv(url)`	Quando o portal publica CSV diretamente
API com CSV	`pd.read_csv(api)`	Quando a API já entrega tabela pronta
API JSON	`resp.json()`	Quando é preciso ler uma estrutura hierárquica

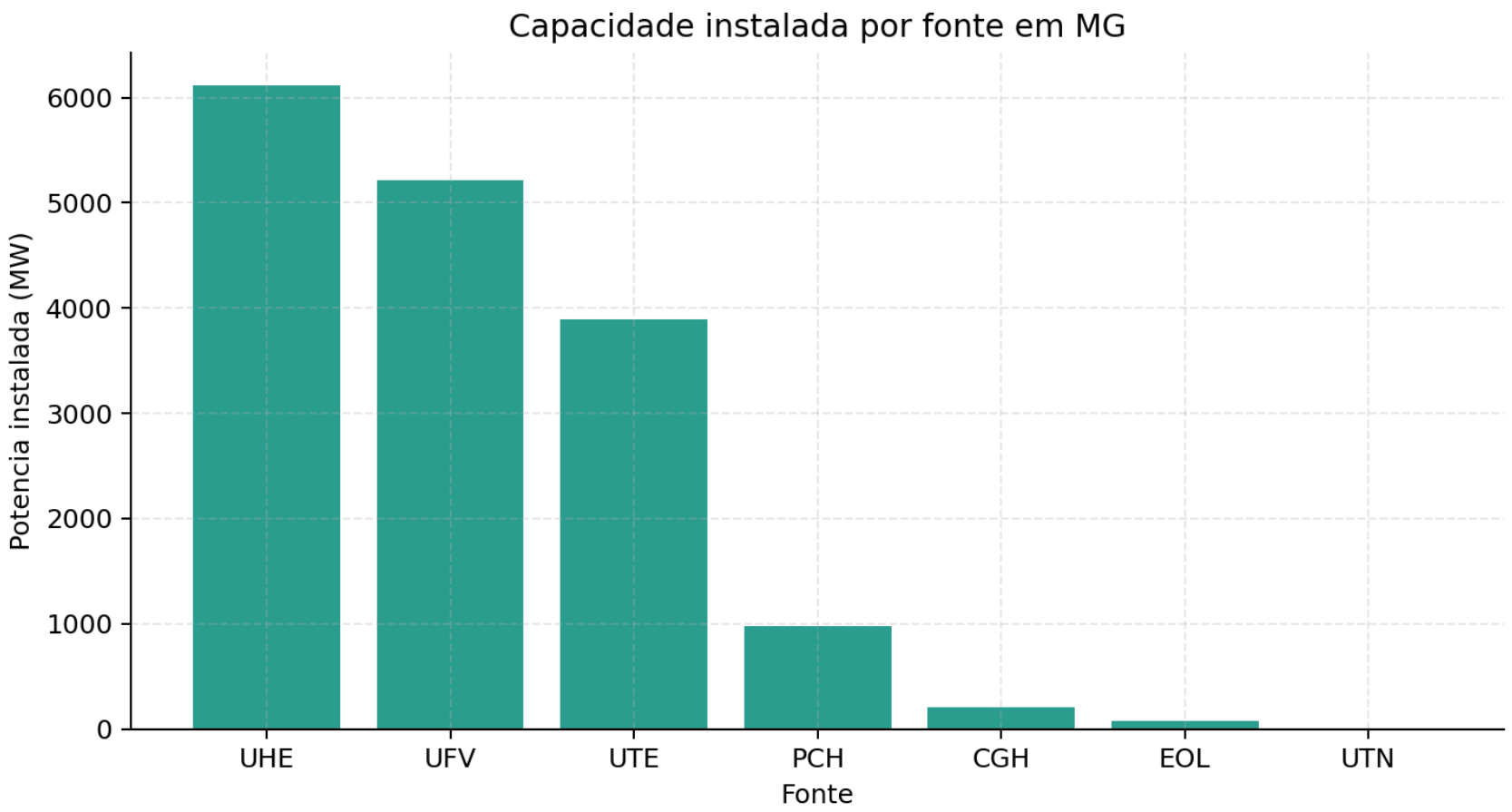
Em simplicidade:

Arquivo local, arquivo remoto, API em CSV e só depois API JSON.

[Exemplo 1] ONS: Fonte oficial para estudos de demanda e operação do SIN



[Exemplo 2] ANEEL e o parque gerador



APIs

O que é uma API?

Definição prática

API (*Application Programming Interface*) é uma forma padronizada de um sistema entregar dados ou serviços para outro programa.

- Para nós, a API funciona como uma porta de acesso organizada aos dados.
- Em vez de copiar dados manualmente do navegador, o Python faz a requisição diretamente.
- A resposta costuma vir em formatos estruturados, como JSON ou CSV.

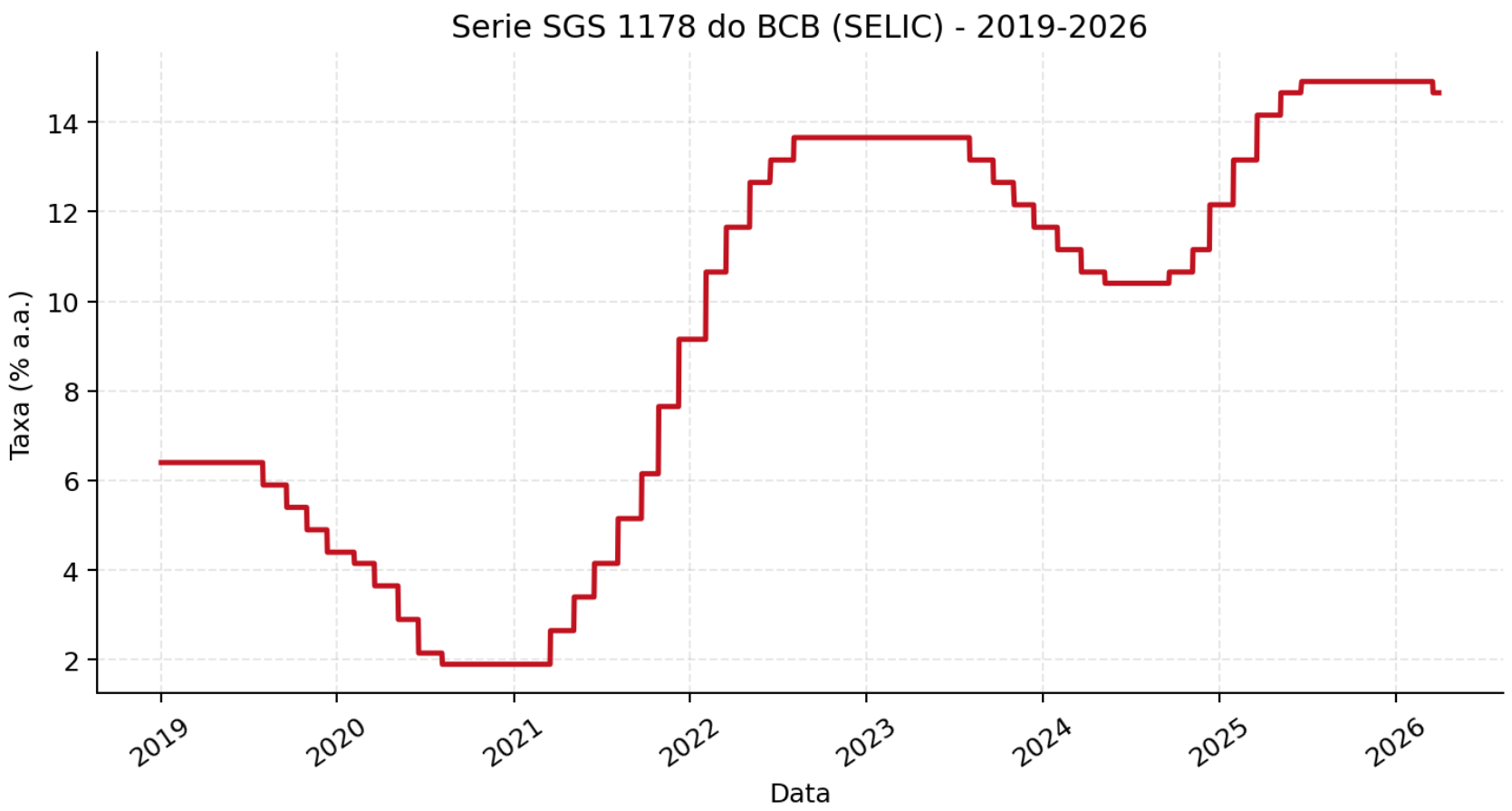
Analogia: a API funciona como um cardápio; você não entra na cozinha, apenas faz um pedido em um formato que o sistema entende.

API em 3 passos

```
1 url = "https://servicodados.ibge.gov.br/"
2       "api/v1/localidades/estados?orderBy=nome"
3 resposta = requests.get(url, timeout=20)
4 estados = resposta.json()
5 df = pd.json_normalize(estados)
```

- Este é o fluxo mais simples: URL, requisição, tabela.
- Se a resposta vier em CSV, basta usar `pd.read_csv(url)`.

[Exemplo 3] Taxa SELIC via API



Anatomia de uma chamada

Elemento	Papel
URL base	Define o serviço principal
Parametros	Filtram período, formato, país, indicador etc.
Status code	Informa sucesso ou falha da requisição
Corpo da resposta	Traz dados em JSON, CSV, XML ou HTML

O que ainda importa saber sobre APIs

- Nem toda API é aberta: algumas exigem chave, token ou autenticação;
- a consulta pode falhar por timeout, erro de rede ou indisponibilidade temporária;
- parâmetros precisam ser lidos na documentação;
- a resposta deve ser inspecionada antes do uso: colunas, tipos, datas e valores ausentes;
- muitas APIs possuem limites de uso ou mudanças de versão ao longo do tempo.

Atenção

Uma URL que funciona hoje não garante que continuará funcionando da mesma forma no futuro.

Boas práticas em Machine Learning

- Salvar o dado bruto antes do tratamento.
- Registrar URL, data de coleta, unidades e observações.
- Validar colunas esperadas e tipos de dados.
- Separar claramente dado bruto, dado tratado e figuras.
- Começar do caso mais simples: arquivo local, arquivo remoto, depois API.
- Não confiar em links para o longo prazo. Mantenha o dado offline sempre que possível.

Encerramento

Síntese

- Ler um arquivo não é apenas abrir um dado: é interpretar estrutura, tipos, unidades e contexto físico.
- Fontes confiáveis pedem documentação, metadados e rastreabilidade.
- Dados abertos e APIs transformam coleta repetitiva em processo reprodutível.
- O notebook desta aula demonstra todos os exemplos mostrados nos slides.

Exercícios sugeridos I

1. A partir de `leituras_bancada.csv`, calcule uma estimativa de potência elétrica e identifique os instantes em que o sistema opera em condição de atenção ou crítica.
2. Trate o valor ausente de temperatura da bancada com duas estratégias diferentes e compare o impacto nas estatísticas descritivas.
3. Use `sensores.json` para montar uma tabela com sensor, grandeza, faixa de medição, unidade e data de calibração.

Exercícios sugeridos II

1. Troque o subsistema da curva de carga do ONS, compare o horário de pico e discuta quais variáveis externas poderiam ajudar em um modelo de previsão.
2. Filtre a amostra da ANEEL para outra UF, identifique a fonte dominante e discuta o que essa composição indica para expansão e operação do sistema.
3. Procure, em OpenML, Kaggle, NASA ou Mendeley Data, uma base ligada a falhas em máquinas, baterias, qualidade de energia ou sensores, e justifique sua adequação a um problema machine learning.
4. Escolha uma API simples, construa a consulta em Python e registre a procedência do dado: fonte, URL, data de coleta, variáveis e formato de resposta.

Referências I

- GRUS, Joel. *Data Science do Zero: Primeiras Regras com o Python*. 2a ed. Alta Books, 2020.
- ONS Dados Abertos. Curva de Carga Horária. `https://dados.ons.org.br/dataset/curva-carga`
- ANEEL Dados Abertos. SIGA. `https://dadosabertos.aneel.gov.br/dataset/siga-sistema-de-informacoes-de-geracao-da-aneel`
- dados.gov.br. Portal de dados abertos do governo federal. `https://dados.gov.br/home`
- IBGE. API de serviço de dados. `https://servicodados.ibge.gov.br/api/docs/`
- Banco Central do Brasil. BCData/SGS. `https://dadosabertos.bcb.gov.br/en/dataset/27803-sgs`

Referências II

- OpenML. `https://www.openml.org/`
- Kaggle Datasets. `https://www.kaggle.com/datasets`
- NASA Open Data. `https://data.nasa.gov/`
- EIA Open Data. `https://www.eia.gov/opendata/`
- FRED. `https://fred.stlouisfed.org/`
- Google Dataset Search. `https://datasetsearch.research.google.com/`
- Mendeley Data. `https://data.mendeley.com/`